

大作业报告

主要参考：*Amazons Discover Monte-Carlo* Richard J. Lorentz 以及网络上学长的代码，参考了结构与一些细节。

代码分为 `Head.h`，`Board.h`，`main.cpp` 三部分：`Head.h` 存放了蒙特卡洛算法使用的参数和符合个人代码风格的宏定义等内容，包括部分数据类型简写；`Board.h` 是一个棋盘类，包括了棋盘的基本操作和蒙特卡洛算法；`main.cpp` 则实现了一个简陋的 UI，支持读档、存档、悔棋、当然还有对弈，可以选择先后手。

另外，还有一份 `bot.cpp` 同时提交，作为 botzone 上 bot 的留档。

在实现基础棋盘功能时，选择了基本的数字矩阵存储，用四种不同的数分别代表黑棋，白棋，障碍，空格。同时存储棋子位置使用了十进制压位，表示走法也使用了十进制压位。使用二进制压位可能会在空间上更优，但并无必要且十进制压位时更加简便。

在检查操作合法性时，由于分类讨论情况过多，判掉特殊情况后选择搜索出所有合法走法，判断用户走法是否在合法走法内即可。

在处理悔棋时，采用了存储所有运行过程中棋盘的方法，悔棋只需要丢掉存储的最后两个元素即可，这样也支持了多次悔棋。同时也处理了存档功能。

蒙特卡洛算法部分，采用了 Queenmove、Kingmove 形式估价，最终估价依赖了 Queenmove、Kingmove 导出的四个参数，以及 Mobility 这个参数。

随机则采用了先随机一个棋子和一个走的方向，再选择一个合法的走法；在实现的时候为了防止对应走法无解，采用了先随机一个排列再按顺序枚举的写法，这样可以保证随机不重复。

棋盘类函数功能：

- `void Show()`：展示棋盘，用于查错和 UI。
- `void Init()`：初始化棋盘。
- `bool In(int x,int y)`：检查 (x,y) 坐标是否在棋盘内。
- `bool Check_Move(int sx,int sy,int ex,int ey,int bx,int by,bool op)`：检查操作 $(s_x, s_y, e_x, e_y, b_x, b_y)$ 是否合法，*op* 表示是否输出错误信息，用于 UI。
- `void Move(int sx,int sy,int ex,int ey,int bx,int by,int col)`：进行移动 $(s_x, s_y, e_x, e_y, b_x, b_y)$ ，同时 *col* 表示这一步棋子的颜色。
- `bool Search_Move(int x,int y)`：对 (x,y) 位置的棋子寻找走法，这里只需要枚举第一步就行，不需要枚举放障碍。
- `int Board_Judge()`：终局判断，做法是枚举每一个棋子做 `Search_Move`，返回值表示哪一方的棋子无法移动。
- `int calc_Move(int x,int y)`：计算 (x,y) 位置的棋子有多少种走法。
- `void calcSol()`：利用 `calc_Move`，判断 UCTTree 上的节点是否还有儿子未扩展到。

- `void calc_Queenmove(int col,int dist[8][8])` 和 `void calc_Kingmove(int col,int dist[8][8])`：计算 Queenmove 和 Kingmove 下的距离，用于 `pure_eval`。
- `int randSol(Board *x,int col)`：随机走法，具体细节在上文有提到。
- `bool noSon()` 和 `bool isLeaf()` 前者判定当前节点还有没有需要扩展的儿子，后者判定当前节点是否是叶子，用于 `Search` 一开始的寻找节点。
- `void copy(Board fa)`：复制 `fa` 的信息到当前节点。
- `db pure_eval()`：对一个局面从黑方的角度进行估价，估价的细节上文有提到。
- `db eval()`：对一个局面先随机模拟再进行估价。
- `Board* Bestfornow()`：寻找一个局面在 `UCTTree` 上子节点的最优局面。
- `Board* expand()`：对一个局面进行随机的扩展并估价。
- `void Search()`：构建一棵 `UCTTree`。

剩下的函数并未用到，故不再赘述。